

SistemaNoleggi -

Documentazione Finale

Ragozzini Nicola DE1000042

Russo Gaetano DE1000051

Descrizione delle classi principali

Auto

La classe **Auto** rappresenta un veicolo disponibile per il noleggio. Gli attributi principali sono **idAuto**, **targa**, **modello**, **statoAuto** e **costoDaily**, che identificano il veicolo, ne descrivono lo stato e il costo giornaliero del noleggio. I metodi principali consentono di visualizzare e modificare i dati dell'auto, aggiornare lo stato (ad esempio da *Disponibile* a *Noleggiata*) e calcolare il costo del noleggio in base ai giorni. L'auto è coinvolta nella relazione con la **Prenotazione**, poiché una prenotazione riguarda sempre una specifica auto.

Utente

La classe **Utente** rappresenta la classe base del sistema per tutti gli utenti registrati. Contiene gli attributi comuni **idUtente**, **username**, **passwordHash**, **email**, **nome** e **cognome**. Fornisce i metodi per l'autenticazione e la gestione delle informazioni personali. Da questa classe derivano **Cliente** e **Operatore**, evitando duplicazioni degli attributi comuni.

Cliente

La classe **Cliente** estende **Utente** e rappresenta l'utente che può effettuare prenotazioni. Gli attributi aggiuntivi sono **patente** e **credito**. Le funzionalità principali permettono di effettuare una prenotazione, consultare il credito disponibile e gestire i propri noleggi. Un cliente può effettuare più prenotazioni nel tempo.

Operatore

La classe **Operatore** estende **Utente** e identifica il personale che gestisce il sistema. L'attributo principale è **ruolo**, appartenente all'enumerazione **Ruolo** (*ADMIN*, *ADDETTONOLEGGIO*, *MANUTENTORE*). Un operatore può confermare o annullare prenotazioni, aggiornare lo stato delle auto e svolgere le operazioni previste dal proprio ruolo.

Prenotazione

La classe **Prenotazione** rappresenta la richiesta di noleggio effettuata da un cliente. Gli attributi sono **idPrenotazione**, **dataInizio**, **dataFine** e **statoPren**, appartenente all'enumerazione **StatoPren** (*IN_ATTESA*, *CONFERMATA*, *ANNULLATA*). I metodi principali permettono di confermare, annullare e consultare lo stato della prenotazione. Ogni prenotazione è associata a un cliente e a una specifica auto e, se confermata, genera un noleggio.

Noleggio

La classe **Noleggio** rappresenta il noleggio effettivo derivante da una prenotazione confermata. Contiene gli attributi **idNoleggio**, **dataRitiro**, **dataRestituzione** e **costoTotale**. È possibile registrare il ritiro e la restituzione dell'auto e calcolare il costo complessivo del

noleggio. Ogni noleggio è associato a una prenotazione e prevede un pagamento.

Pagamento

La classe **Pagamento** rappresenta il pagamento relativo a un noleggio. Gli attributi sono **idPagamento**, **importo** e **statoPagamento**, appartenente all'enumerazione **StatoPagamento** (*IN_ATTESA*, *COMPLETATO*, *FALLITO*). I metodi permettono di registrare il pagamento, verificarne lo stato e aggiornarlo in seguito all'esito della transazione. Ogni pagamento è riferito a un solo noleggio.

Enumerazioni

Ruolo

L'enumerazione **Ruolo** definisce i possibili ruoli degli operatori del sistema:

- **ADMIN**: amministratore del sistema.
 - **ADDETTONOLEGGIO**: gestisce prenotazioni e noleggi.
 - **MANUTENTORE**: aggiorna lo stato delle auto in manutenzione.
-

StatoPren

L'enumerazione **StatoPren** indica lo stato di una prenotazione:

- **IN_ATTESA**: prenotazione appena effettuata.
- **CONFERMATA**: prenotazione accettata.
- **ANNULLATA**: prenotazione cancellata.

StatoAuto

L'enumerazione **StatoAuto** descrive la disponibilità di un veicolo:

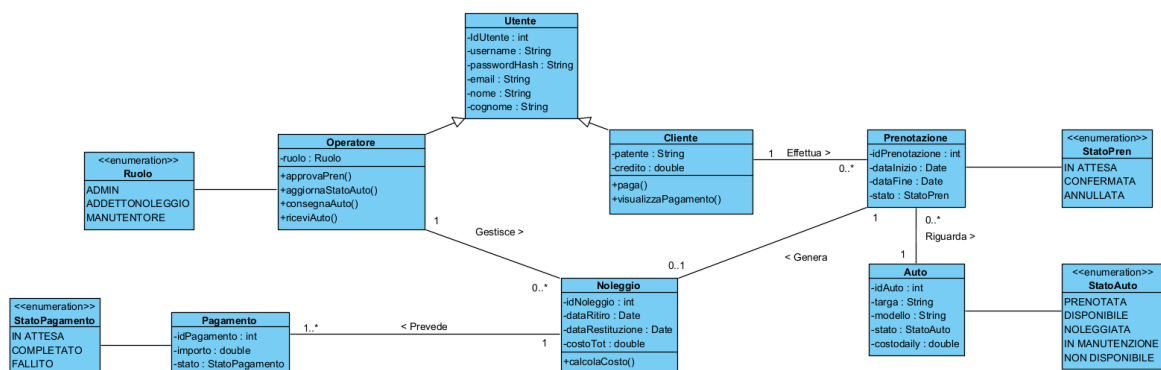
- DISPONIBILE
 - PRENOTATA
 - NOLEGGIATA
 - IN_MANUTENZIONE
 - NON_DISPONIBILE
-

StatoPagamento

L'enumerazione **StatoPagamento** rappresenta lo stato del pagamento:

- IN_ATTESA
- COMPLETATO
- FALLITO

Class Diagramm : model



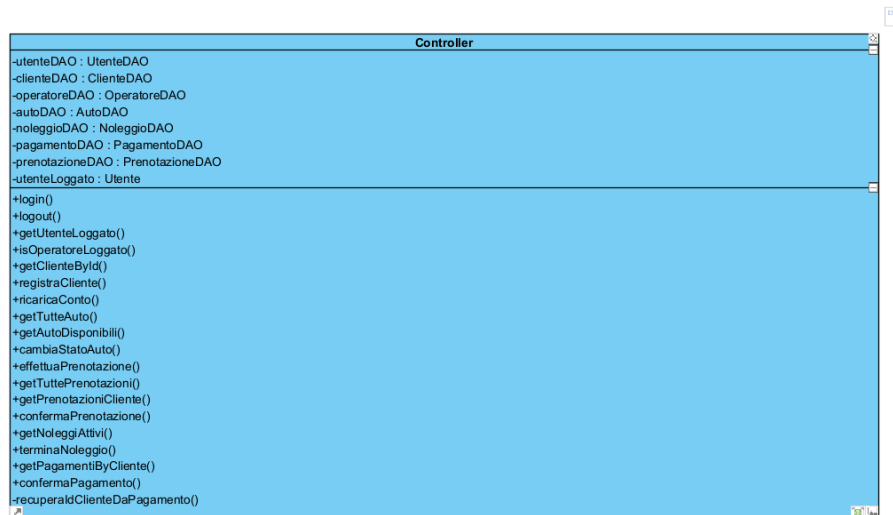
Controller

La classe **Controller** rappresenta il livello di controllo dell'applicazione e funge da intermediario tra la GUI e il livello di accesso ai dati (DAO). Contiene i riferimenti a tutte le interfacce DAO necessarie alla gestione del sistema e mantiene l'informazione relativa all'utente attualmente autenticato.

I principali metodi permettono di:

- autenticare gli utenti (login, logout);
- registrare nuovi clienti;
- gestire il credito del cliente;
- visualizzare e modificare lo stato delle auto;
- effettuare e confermare prenotazioni;
- creare e terminare i noleggi;
- gestire i pagamenti.

Class Diagramm : Controller



Package dao

Interfacce DAO

Le interfacce del package **dao** definiscono le operazioni di accesso ai dati delle diverse entità del sistema. Ogni interfaccia dichiara i metodi CRUD (creazione, ricerca, modifica ed eliminazione) senza specificarne l'implementazione. In questo modo il controller rimane indipendente dal database utilizzato.

Package implementazionePostgresDAO

Implementazioni DAO

Le classi del package **implementazionePostgresDAO** implementano le interfacce DAO utilizzando PostgreSQL tramite JDBC. Esse eseguono le query SQL necessarie per salvare, modificare, eliminare e recuperare i dati dal database, trasformando i risultati delle query negli oggetti del modello.

ImpUtenteDAO

La classe **ImpUtenteDAO** implementa l'interfaccia **UtenteDAO** e gestisce la persistenza degli utenti. I principali metodi permettono di salvare, cercare, aggiornare ed eliminare utenti. Durante il recupero dei dati riconosce automaticamente se l'utente è un **Cliente**, un **Operatore** o un semplice **Utente**, ricostruendo l'oggetto corretto.

ImpClienteDAO

La classe **ImpClienteDAO** implementa **ClienteDAO** e gestisce le informazioni specifiche dei clienti. I metodi principali consentono di registrare clienti, aggiornarne i dati, modificare il credito disponibile e recuperare uno o più clienti dal database.

ImpOperatoreDAO

La classe **ImpOperatoreDAO** implementa **OperatoreDAO** e gestisce la persistenza degli operatori. Permette di salvare, cercare, aggiornare ed eliminare operatori, memorizzando anche il ruolo associato a ciascun operatore.

ImpAutoDAO

La classe **ImpAutoDAO** implementa **AutoDAO** e gestisce le operazioni sulle auto presenti nel sistema. I principali metodi

permettono di inserire, modificare, eliminare e ricercare le auto, oltre ad aggiornare il loro stato e recuperare l'elenco delle auto disponibili per il noleggio.

ImpPrenotazioneDAO

La classe **ImpPrenotazioneDAO** implementa **PrenotazioneDAO** e gestisce la persistenza delle prenotazioni. I suoi metodi consentono di creare, aggiornare, eliminare e ricercare prenotazioni, ricostruendo le relazioni con le entità **Cliente** e **Auto** attraverso opportune query SQL.

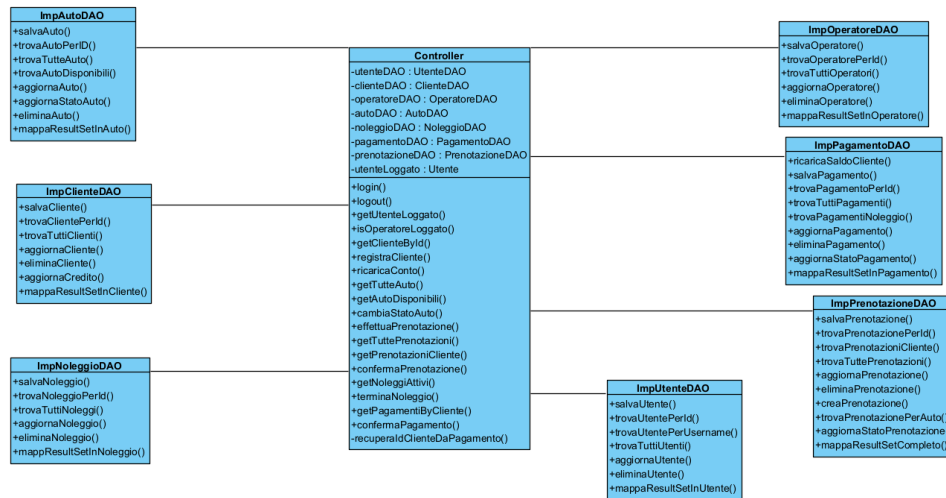
ImpNoleggioDAO

La classe **ImpNoleggioDAO** implementa **NoleggioDAO** e gestisce la memorizzazione dei noleggi. Oltre alle normali operazioni CRUD, ricostruisce mediante query con JOIN le relazioni tra **Noleggio**, **Prenotazione**, **Cliente** e **Auto**, permettendo di ottenere oggetti completi.

ImpPagamentoDAO

La classe **ImpPagamentoDAO** implementa **PagamentoDAO** e gestisce la persistenza dei pagamenti. I metodi principali permettono di registrare un pagamento, aggiornarne lo stato, recuperare i pagamenti associati a un cliente o a un noleggio e gestire la ricarica del credito dei clienti.

Class Diagramm : ImplementazioniDao



ConnessioneDatabase

La classe **ConnessioneDatabase** centralizza la gestione della connessione al database PostgreSQL.

Attraverso il metodo statico `getConnection()` crea e restituisce una connessione JDBC utilizzata da tutte le implementazioni DAO, evitando duplicazioni di codice e semplificando la manutenzione dell'applicazione.

Main

La classe **Main** rappresenta il punto di ingresso dell'applicazione.

Nel metodo `main()` vengono istanziate le implementazioni delle interfacce DAO, inizializzato il **Controller** e avviata l'interfaccia grafica tramite la finestra di login. In questo modo vengono collegati tutti i livelli dell'architettura (GUI, Controller e Data Access Layer) prima dell'avvio del sistema.

Package gui

Le classi all'interno del package `gui` gestiscono l'interfaccia grafica dell'applicazione, cioè tutto quello che l'utente vede sullo schermo e con cui interagisce. Seguendo la struttura del progetto, queste classi non toccano mai direttamente il database: quando l'utente clicca su un pulsante, la grafica cattura i dati e li passa al `Controller`, che si occupa di fare il lavoro vero e proprio. Inoltre, la grafica cambia in automatico in base a chi entra nel programma (Cliente o Operatore), mostrando o nascondendo i pulsanti.

LoginFrame

È la prima finestra che si apre quando si avvia il programma e serve per entrare nel sistema.

- **Descrizione:** Mostra due caselle di testo dove l'utente deve inserire lo username e la password.
- **Funzionalità principali:** Quando si clicca sul tasto di login, manda le credenziali al `Controller` per verificare se sono giuste. Se l'accesso va a buon fine, chiude la finestra di login per non sprecare memoria e apre la schermata principale del programma (`DashboardFrame`). Contiene anche un pulsante per aprire la finestra di registrazione se l'utente è nuovo.

RegistrazioneFrame

È la finestra che permette ai nuovi clienti di registrarsi.

- **Descrizione:** Contiene un modulo dove inserire tutti i dati personali.
- **Funzionalità principali:** Controlla che nessun campo (Nome, Cognome, Email, Username, Password, Patente) sia stato lasciato vuoto. Poi crea un nuovo oggetto `Cliente` con il credito iniziale a zero e dice al `Controller` di salvarlo nel database.

AutoPanel

È il pannello principale per vedere e gestire le automobili.

- **Descrizione:** Mostra una tabella con l'elenco delle auto e i loro dettagli (ID, targa, modello, costo al giorno e stato attuale). La tabella è protetta e non si può modificare cliccandoci sopra.
- **Funzionalità principali:** Si adatta da sola in base a chi è loggato. Se l'utente è un operatore, nasconde il tasto per prenotare e mostra i tasti per mettere l'auto "In Manutenzione" o farla tornare "Disponibile". Se l'utente è un cliente, mostra il tasto per prenotare e apre una finestrella per chiedere fino a quale giorno si vuole tenere l'auto.

ClientePanel

È la schermata dedicata al profilo personale del cliente.

- **Descrizione:** Mostra un riepilogo con il nome del cliente, la sua email e i soldi che ha ancora a disposizione (il credito).
- **Funzionalità principali:** Ha una casella di testo e un pulsante che permettono al cliente di ricaricare il proprio conto. Per evitare errori con i centesimi, i calcoli usano un formato matematico preciso (`BigDecimal`). Inoltre, ogni volta che il

cliente apre questa schermata, il pannello si aggiorna da solo andando a riprendere i dati aggiornati dal database.

OperatorePanel

È il pannello pensato appositamente per i dipendenti e i tecnici del noleggio.

- **Descrizione:** Mostra la tabella con tutte le auto presenti nel sistema.
- **Funzionalità principali:** Permette all'operatore di controllare rapidamente lo stato di ogni veicolo e di cambiarlo con un click (ad esempio per inviare un'auto in riparazione o rimetterla in servizio), aggiornando subito i dati per tutti gli utenti.

PrenotazionePanel

Questo modulo serve a tenere d'occhio tutte le richieste di prenotazione.

- **Descrizione:** Mostra una tabella con le date di inizio e fine noleggio, il modello dell'auto scelta e lo stato della richiesta (ad esempio "In Attesa").
- **Funzionalità principali:** Se l'utente è un cliente, vede soltanto le prenotazioni fatte da lui. Se invece è un operatore, vede la lista completa di tutti i clienti e ha a disposizione il pulsante per confermare e accettare le prenotazioni in sospeso.

NoleggioPanel

È il pannello che serve a controllare i noleggi che sono in corso in questo momento.

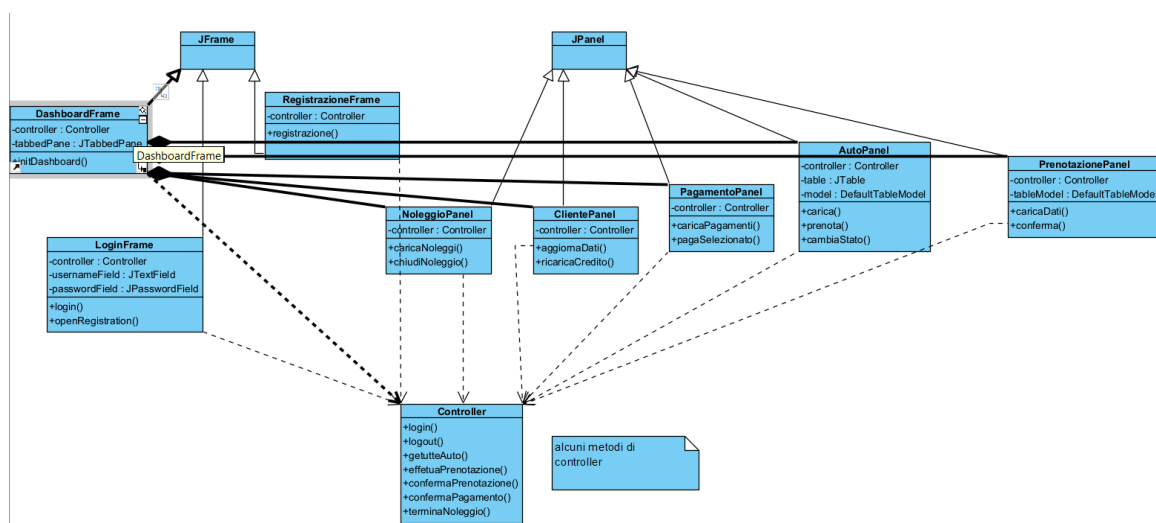
- **Descrizione:** Mostra le auto che sono state effettivamente ritirate dai clienti e che stanno viaggiando.
- **Funzionalità principali:** Permette agli operatori di vedere chi sta usando una determinata auto e offre un pulsante per terminare il noleggio quando il cliente riporta indietro la vettura. Questa operazione libera l'auto e fa calcolare il prezzo finale.

PagamentoPanel

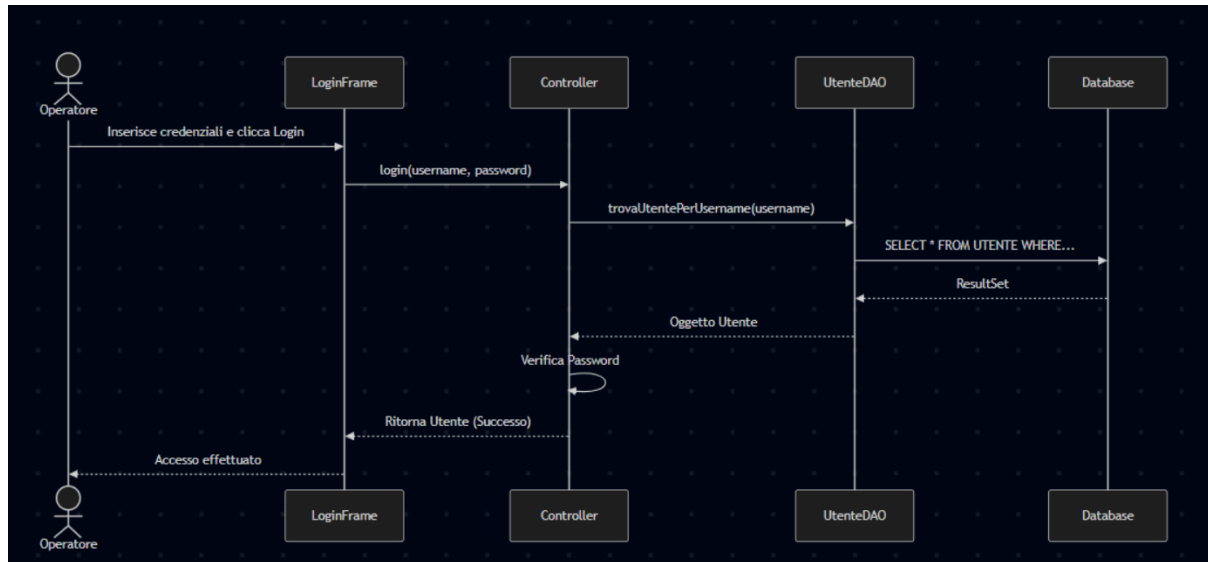
Gestisce la parte economica e i pagamenti dal punto di vista del cliente.

- **Descrizione:** Mostra una lista con i prezzi da pagare o già pagati e lo stato della transazione.
- **Funzionalità principali:** Trova in automatico i pagamenti del cliente loggato e gli permette di selezionare una voce in sospeso e cliccare su un pulsante per pagarla usando il credito del suo conto.

Class Diagramm : GUI



SEQUENCE DIAGRAMM: LOGIN



Descrizione del Processo di Login

Il diagramma mostra come funziona il sistema quando un utente fa il login, usando una struttura divisa in tre parti: l'interfaccia grafica, la logica dei controlli e il database. Tutto inizia quando l'**Operatore** inserisce username e password nella schermata di accesso (**LoginFrame**) e clicca il pulsante per entrare. A questo punto, il **LoginFrame** prende i dati e li passa al **Controller** tramite il comando `login(username, password)`. Il **Controller** chiede aiuto all'**UtenteDAO** usando il comando `trovaUtentePerUsername(username)`. L'**UtenteDAO** fa quindi una ricerca vera e propria dentro al **Database** con la query

`SELECT * FROM UTENTE WHERE . . .` Il **Database** risponde mandando indietro i dati grezzi trovati (il `ResultSet`), che l'**UtenteDAO** trasforma in un normale **Oggetto Utente** leggibile dal programma, rispedendolo al **Controller**. Ora il **Controller** fa il controllo più importante: con l'operazione interna

Verifica Password che confronta la password scritta dall'utente con quella salvata nel database. Se i dati corrispondono, il **Controller** comunica il successo al **LoginFrame** e infine la schermata si aggiorna mostrando all'**Operatore** il messaggio di conferma Accesso effettuato.

Considerazioni sulle scelte progettuali

Per questo progetto stato introdotto un unico Controller centralizzato che fa da mediatore: le schermate della gui non implementano logiche complesse, ma chiamano direttamente questo componente, il quale a sua volta interroga subito gli oggetti del dominio (model) e i componenti di accesso ai dati (DAO). Questa scelta riduce al minimo i passaggi intermedi, mantenendo il codice snello e facile da seguire.

Il riuso del codice è stato affrontato principalmente attraverso l'ereditarietà lato dati. Le classi Cliente e Operatore estendono la classe base Utente, ereditando in automatico tutti gli attributi comuni come nome, cognome ed email, evitando inutili duplicazioni nel modello.

Infine, l'estensibilità del sistema è garantita dall'uso delle interfacce per lo strato DAO (come AutoDAO o ClienteDAO). Poiché il Controller fa riferimento a queste interfacce astratte e non alle classi concrete, se in futuro si decidesse di cambiare database (passando ad esempio da PostgreSQL a un altro sistema), basterà creare le nuove classi di implementazione senza dover toccare una sola riga di codice né nel Controller né nei pannelli grafici.

